TASK 2 — STEP-BY-STEP GUIDE

IDENTIFY USE CASES · WORKED EXAMPLES & STUDENT PRACTICE

Course: Geo Programming | Week 5 | Suggested time: 15–25 minutes | Individual work

1. Learning Outcomes

Identify actors and goals; derive at least six use cases; describe actor, precondition, main flow and expected result; model error flows; and produce a use-case table or diagram linked to Task 1 requirements.

2. Problem Statement — Review Before Starting

For academic year 2026–2027, a school uses an iOS application to manage students, teachers, subjects and enrollments. Staff can browse records, add/edit students, assign subjects to teachers, enroll students, enter/update marks (0–10), and inspect student results. Duplicate enrollment in the same subject and academic year is not allowed. A teacher may teach several subjects, and a subject may have several teachers.

Important: Task 1 requirements are your input. Focus on WHAT users achieve, not Swift class names or screen implementation.

3. Guided Procedure

Step 1 — Identify Actors and Goals (2 minutes)

An actor is a role interacting with the system. The specification explicitly says “staff”; specialized roles below are proposed assumptions, not stated facts.

Example	Actor	Goal	Status
1	School staff	Browse student records	Specified
2	School staff	Enroll a student in a subject	Specified
3	Academic coordinator	Assign a subject to a teacher	Assumed specialization of staff

YOUR TURN: Propose 3–5 additional actor–goal pairs. Mark every new role as specified or assumed.

No.	Actor / role	Goal	Specified / assumed?
1	School staff	Search students by ID or name	Specified (browse records)
2	School staff	Add a new student	Specified
3	School staff	Edit a student's name and email	Specified
4	School staff	Inspect a student's subjects and marks	Specified
5	Academic coordinator	Assign subjects to teachers; browse subjects and teachers	Assumed specialization of staff

Step 2 — Convert Functional Requirements into Use Cases (3 minutes)

Start with an FR from Task 1, then express the user goal as a verb + object. One use case may cover multiple related requirements.

Example	Task 1 FR (illustrative)	Use case
1	FR-01: Display student records	UC-01 — Browse Students
2	FR-04: Enroll a student	UC-02 — Enroll Student
3	FR-06: Update a mark	UC-03 — Update Student Mark

YOUR TURN: Derive 3–5 additional use cases from your own Task 1 FRs. Use unique UC identifiers.

FR ID(s)	Use-case ID	Verb + object name
FR-02	UC-04	Search Students
FR-03, FR-12	UC-05	Add Student
FR-04, FR-12	UC-06	Edit Student
FR-06	UC-07	Assign Subject to Teacher
FR-11	UC-08	Inspect Student Results
FR-05	UC-09	Browse Subjects and Teachers

Step 3 — Describe Each Use Case (5 minutes)

For each use case, define actor, precondition, main flow (numbered actions), and expected result. The examples are abbreviated to demonstrate the format.

UC / actor	Precondition	Main flow	Expected result
UC-01 Browse Students / Staff	Student records exist	1. Open student records. 2. Request list. 3. Review results.	Student IDs and names are displayed.
UC-02 Enroll Student / Staff	Student and subject exist	1. Choose student. 2. Choose subject and year. 3. Confirm.	One valid enrollment is recorded.
UC-03 Update Mark / Staff	Enrollment exists	1. Find enrollment. 2. Enter mark. 3. Save.	Updated mark is shown.

YOUR TURN: Write 3–5 complete use-case descriptions. For each, include at least three numbered main-flow actions.

UC ID / actor	Precondition	Main flow (3+ actions)	Expected result
UC-04 / Staff	Student list is open	1. Tap the search field. 2. Type part of an ID or name. 3. Review the filtered list.	Only matching students are shown; “No matching students” if none.
UC-05 / Staff	Students screen is open	1. Tap Add (+). 2. Enter student ID, name and email. 3. Tap Save.	The new student is stored and appears in the list.
UC-06 / Staff	The student exists	1. Open the student. 2. Tap Edit. 3. Change name or email. 4. Tap Save.	Updated values are shown in the detail and the list.
UC-07 / Academic coordinator (assumed)	Teacher and subject exist	1. Open the teacher. 2. Tap “Assign a subject”. 3. Choose a subject. 4. Tap Assign.	The subject appears in the teacher's subject list.
UC-08 / Staff	The student exists	1. Open Students. 2. Tap a student. 3. Read the subjects, marks and average.	Enrolled subjects, marks and average mark are displayed.
UC-09 / Staff	Subjects and teachers exist	1. Open the Subjects or Teachers tab. 2. Tap a row. 3. Read the details.	Subject info with its teachers and students (or a teacher's subjects) is shown.

Step 4 — Define Alternate and Error Flows (2 minutes)

An alternate/error flow starts at a specific step in the main flow and explains how the app responds to a failed condition.

Example	Trigger / branch	Expected system response
1 — Duplicate enrollment	UC-02, step 3: same student + subject + year already exists	Reject duplicate; explain why; keep original record.
2 — Invalid mark	UC-03, step 2: mark = 11	Show validation error; do not change saved mark.
3 — Missing student ID	Add Student: required ID is empty	Request valid ID; do not create record.

YOUR TURN: Add 3–5 alternate/error flows. For each, identify the related UC, the triggering step, and the expected response.

Related UC / step	Error condition	System response
UC-05, step 3	Student ID already exists (e.g. S001)	Show “ID S001 already exists.”; do not create the record.
UC-05 / UC-06, step 3	Email is not valid (e.g. “kiet@”)	Show “Please enter a valid email”; the record is not created or changed.
UC-07, step 4	The subject is already assigned to this teacher	Show “This subject is already assigned to this teacher.”; assignments unchanged.
UC-02, step 2	Academic year in the wrong format (e.g. “2026”)	Show “Academic year must look like 2026-2027.”; no enrollment is created.
UC-04, step 2	No student matches the search text	Show the empty state “No matching students”.

Step 5 — Draw a Use-Case Diagram or Create a Table (2 minutes)

Choose a use-case table or a UML-style diagram. Show actor-to-use-case associations and use-case IDs. Do not show classes, database tables or implementation methods.

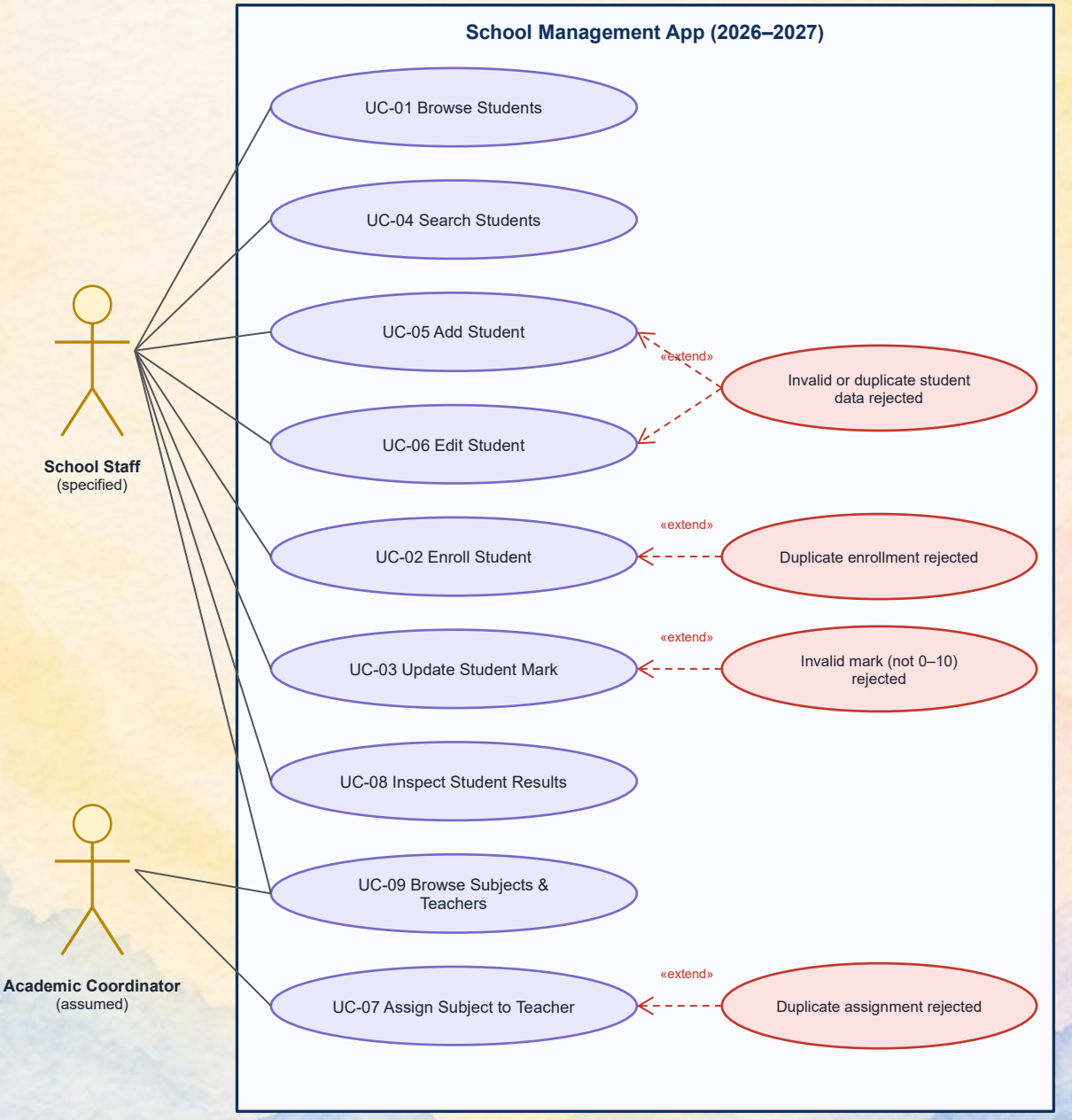
Example	Actor → use case	Requirement trace

1	Staff → Browse Students	UC-01 → FR-01
2	Staff → Enroll Student	UC-02 → enrollment FR
3	Staff → Update Student Mark	UC-03 → mark-update FR

YOUR TURN: Add 3–5 more associations and trace each one to a Task 1 FR. Then draw one diagram OR submit the completed table.

Actor	Use case	FR ID(s)
Staff	Search Students (UC-04)	FR-02
Staff	Add Student (UC-05)	FR-03, FR-12
Staff	Edit Student (UC-06)	FR-04, FR-12
Academic coordinator (assumed)	Assign Subject to Teacher (UC-07)	FR-06
Staff	Inspect Student Results (UC-08)	FR-11
Staff / coordinator	Browse Subjects and Teachers (UC-09)	FR-05

Use-case diagram (UC-01 → UC-09, with error flows as «extend»):



Step 6 — Review, Trace and Submit (1 minute)

Check each use case for clarity, actor participation and testability. These are sample review checks:

Example check	What to verify
1 — Actor	Every UC has an actor and an observable goal.
2 — Main flow	Steps follow a logical sequence and end with a result.

3 — Traceability	Each UC maps to at least one relevant FR from Task 1.

YOUR TURN: Write 3–5 additional review questions and apply them to your own use cases.

Review question	UC(s) checked	Pass / revise
Does every use case have one actor and an observable result?	UC-01 → UC-09	Pass
Does every main flow have at least three numbered actions?	UC-04 → UC-09	Pass
Does every use case trace to at least one Task 1 FR?	UC-01 → UC-09	Pass
Is every business rule (mark 0–10, no duplicate enrollment) covered by an error flow?	UC-02, UC-03	Pass
Are assumed roles clearly labelled as assumptions?	UC-07, UC-09	Pass

4. Student Deliverables

Submit: (1) at least 6 distinct use cases; (2) a use-case description for each with actor, precondition, main flow and expected result; (3) at least one alternate/error flow; (4) one use-case diagram or table; (5) traceability to Task 1 FR IDs. The 3–5 practice entries in each step are learning exercises; consolidate overlapping entries rather than inventing redundant use cases.

Filename: Task2_UseCases_StudentID.pdf. Include the results later in Requirements.pdf for Lab 01.

5. Self-Assessment Checklist

☑ At least 6 meaningful use cases ☑ Actor + precondition + main flow + result for each ☑ Error flow included ☑ Diagram/table complete ☑ FR traceability ☑ Assumed roles clearly labeled

6. Reflection

Which user goal was most difficult to convert into a use case, and why? Answer in 3–5 sentences.

“Assign subjects to teachers” was the most difficult goal to convert, because the relationship is many-to-many and the specification does not say who performs it. I described it from the teacher's point of view (open a teacher, then choose a subject), so the flow stays short and testable. I kept staff as the specified actor and labelled the Academic Coordinator as an assumption. I also added an error flow for assigning the same subject twice, so that one teacher cannot list a subject more than once.